

CS 446 / ECE 449 — Homework 6

your NetID here

Instructions.

- Homework is due **Friday December 5th**, at 11:59 **PM** CST; you have **3** late days in total for **all Homeworks**.
- Everyone must submit individually at gradescope under **HW6** and **HW6-Programming Assignment**.
- The “written” submission at **HW6** **must be typed**, and submitted in any format gradescope accepts (to be safe, submit a PDF). You may use L^AT_EX, markdown, google docs, MS word, whatever you like; but it must be typed!
- When submitting at **HW6**, Gradescope will ask you to **mark out boxes around each of your answers**; please do this precisely!
- Please make sure your NetID is clear and large on the first page of the homework.
- Your solution **must** be written in your own words and you may not consult LLMs. Please see the course webpage for full **academic integrity** information. You should cite any external reference you use.
- We reserve the right to reduce the auto-graded score for **HW6-Programming Assignment** if we detect funny business (e.g., your solution lacks any algorithm and hard-codes answers you obtained from someone else, or simply via trial-and-error with the autograder).
- When submitting to **HW6-Programming Assignment**, only upload `hw6_q1.py`, `hw6_q3_autograd.py`, `hw6_q3_nn.py`, `hw6_q3_optim.py`, and `hw6_q3_utils.py`. Additional files will be ignored.

1. PCA (30 pt)

- (a) (5 pt) Use SVD to compute the principal components ($k = 2$) of this matrix.

$$\begin{bmatrix} 5 & 1 & 1 \\ 1 & 5 & -1 \end{bmatrix}$$

- (b) (2 pt) How would the resulting “principal components” differ if you didn’t center the matrix?

In lecture, we learned that PCA is equivalent to maximizing the variances of datapoint projections and minimizing the error of the projections. In this problem, we will show the equivalences of various interpretations of PCA. To do so, assume you are given a centered data matrix $\mathbf{X} \in \mathbb{R}^{d \times N}$, where each of the N columns $\mathbf{x}^{(i)}$ in $\mathbf{X}$ are d -dimensional datapoints.

- (c) You have a subspace $\mathcal{S} = \{\mathbf{b} + v\mathbf{u} : v \in \mathbb{R}\}$, where $\mathbf{b}$ is a vector of length d ($\mathbf{b} \in \mathbb{R}^d$) and $\mathbf{u}$ is a unit vector ($\mathbf{u}^\top \mathbf{u} = 1$).
- (2 pt) Derive the formula for $\tilde{x}$, the projection of x onto $\mathcal{S}$, in terms of $\mathbf{b}$ and $\mathbf{u}$.
 - (3 pt) Derive the minimal squared distance $\|\tilde{x} - x\|_2^2$ in terms of x , $\mathbf{b}$, and $\mathbf{u}$.
- (d) (3 pt) What is the formula for the mean squared variance of $\mathbf{X}$ in a certain direction? How can you compute the direction of maximal variance of $\mathbf{X}$?
- (e) (5 pt) Let’s prove that the two definitions from lecture—maximizing the variances of the points projected onto a line and minimizing the errors between the points and their projections—are equivalent.
- (f) (5 pt) Now, prove that finding a rank 1 approximation of the matrix $\mathbf{X}$ is equivalent to selecting the subspace $\mathcal{S}$ that maximizes the variances of the projected points.

Hint 1: Consider how to express a rank 1 matrix

Hint 2: Look into the properties of the Frobenius norm.

Since PCA is equivalent to minimizing reconstruction error, a linear autoencoder optimized using gradient descent can effectively recover the subspace spanned by PCA components (up to a rotation/reflection).

- (g) (5 pt) Implement `hw6_q1.py` to empirically verify this. The output of your encoder may not be directly aligned with the subspace spanned by PCA components. However, assuming the autoencoder is trained correctly and its latent space spans the same subspace of the top k PCA components, you can recover the transformation from the encoder output to PCA components by employing simple least squares. After the transformation, the output of your encoder should align with the PCA components. A Jupyter notebook `hw6.ipynb` is provided for you to test your implementation.

Hint: You should not use two separate linear layers to implement the encoder and decoder.

Solution.

2. Variational Auto-Encoders (25 pt)

We are training a variational auto-encoder (VAE). It consists of the following parts: the input vector $\mathbf{x}$, the latent vector $\mathbf{z}$, the encoder that models the probability $q_\phi(\mathbf{z}|\mathbf{x})$, and the decoder that models $p_\theta(\mathbf{x}|\mathbf{z})$. Based on these notations, let's look at several problems:

- (a) We assume the latent vector $\mathbf{z} \in \mathbb{R}^2$ follows a multi-variate Gaussian distribution $\mathcal{N}$. Please compute the output dimension of the encoder $q_\phi(\mathbf{z}|\mathbf{x})$ under the following cases and briefly explain why. (If “output dimension” is not clear enough for you, think of it as “how many real numbers $r \in \mathbb{R}$ are needed to output for the sampling of latent vectors.”)
 - (2 pt) We assume $\mathcal{N}$ has an **identity matrix** as the covariance matrix.
 - (2 pt) We assume $\mathcal{N}$ has an **diagonal matrix** as the covariance matrix.
- (b) We then consider the problems related to the understanding of KL-Divergence.
 - i. (5 pt) Using the inequality of $\log(x) \leq x - 1$, prove that $D_{KL}(p(\mathbf{x}), q(\mathbf{x})) \geq 0$ holds for two arbitrary distributions $p(\mathbf{x})$ and $q(\mathbf{x})$.
 - ii. (7 pt) Consider a binary classification problem with input vectors $\mathbf{x}$ and labels $y \in \{0, 1\}$. The distribution of the ground truth label is denoted as $P(y)$. The expression of $P(y)$ is as Eq 1, where y_{gt} is the ground truth label.

$$P(y = y_{gt}) = 1, P(y = 1 - y_{gt}) = 0 \quad (1)$$

Suppose we are trying to predict the label of $\mathbf{x}$ with a linear model $\mathbf{w}$ and sigmoid function, then the distribution of y is denoted as $Q(y)$ and computed as Eq. 2.

$$Q(y = 0|\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}, \quad Q(y = 1|\mathbf{x}) = \frac{\exp(-\mathbf{w}^\top \mathbf{x})}{1 + \exp(-\mathbf{w}^\top \mathbf{x})} \quad (2)$$

With the above information, compute the KL Divergence between the distributions of $P(y)$ and $Q(y|\mathbf{x})$, specifically $D_{KL}(P(y)||Q(y|\mathbf{x})) = \mathbb{E}_{y \sim P(y)}[\log \frac{P(y)}{Q(y|\mathbf{x})}]$.

Expand your solution to the clearest form. To get full credits, you may only use y_{gt} , $\mathbf{w}$, $\mathbf{x}$ and related constants in your expression.

- (c) VAE is a special branch of generative method in sampling the latent vectors $\tilde{\mathbf{z}}$ from $q_\phi(\mathbf{z}|\mathbf{x})$ instead of directly regressing the values of $\mathbf{z}$. Read an example implementation of VAE at https://github.com/AntixK/PyTorch-VAE/blob/master/models/vanilla_vae.py and answer the following questions:
 - i. (1 pt) Find the functions and lines related to the sampling of $\tilde{\mathbf{z}}$ from $q_\phi(\mathbf{z}|\mathbf{x})$. Specifying the names of the functions and the related lines can lead to full credits. Please note that if your range is too broad (in the extreme case, covering every line in the file) we cannot give your full credit.
 - ii. (5 pt) Suppose our latent variable is $\mathbf{z} \in \mathbb{R}^2$ sampled from a Gaussian distribution with mean $\boldsymbol{\mu} \in \mathbb{R}^2$ and a diagonal covariance matrix $\boldsymbol{\Sigma} = \text{Diag}\{\sigma_1^2, \sigma_2^2\}$. Then another random variable $\boldsymbol{\epsilon} \in \mathbb{R}^2$ is sampled from a Gaussian distribution $\mathcal{N}(0, \mathbf{I})$. Show that $\mathbf{V} = [\sigma_1, \sigma_2]^\top \odot \boldsymbol{\epsilon} + \boldsymbol{\mu}$ follows the same distribution as $\mathbf{z}$. ($\odot$ denotes element-wise product; $\mathcal{N}(0, \mathbf{I})$ denotes the multi-variate Gaussian with zero mean and identity matrix as covariance.)
 - iii. (2 pt) Under the same setting of the Question ii, we can sample the latent vector $\tilde{\mathbf{z}}$ by the process $\tilde{\mathbf{z}} = [\sigma_1, \sigma_2]^\top \odot \tilde{\boldsymbol{\epsilon}} + \boldsymbol{\mu}$, where $\tilde{\boldsymbol{\epsilon}}$ is a sampled random variable from $\mathcal{N}(0, \mathbf{I})$. Consider the process of training, where we apply back-propagation to train the neural networks. Given the gradient on $\tilde{\mathbf{z}}$ as $\tilde{\mathbf{g}} \in \mathbb{R}^2$, which can be written as $[\tilde{g}_1, \tilde{g}_2]^\top$. **What are the gradients of the output of the encoder: $\boldsymbol{\mu}, \sigma_1, \sigma_2$?** (Assume the KL-Divergence loss is not considered in this part.)
Note: To get full credit, you may use any constants and the variables $\tilde{\boldsymbol{\epsilon}} = [\tilde{\epsilon}_1, \tilde{\epsilon}_2]$, $\tilde{\mathbf{g}} = [\tilde{g}_1, \tilde{g}_2]^\top$, and $\boldsymbol{\mu}, \sigma_1, \sigma_2$.
 - iv. (1 pt) After reading the code, explain why we sample $\tilde{\mathbf{z}}$ with $\mathcal{N}(0, 1)$, instead of directly generating the values.

Solution.

3. Neural Networks (29 pt)

In this problem, you will build a simple neural network framework from scratch, featuring a PyTorch-like API. At the core of popular deep learning frameworks such as PyTorch, TensorFlow, JAX, etc, are the automatic differentiation engines that compute gradients of common functions. During training, the losses are backpropagated through the network, and the resulting gradients are used to update the network parameters. You will implement several common functions, such as sigmoid activation and binary cross-entropy, as well as standard layers, such as fully connected (linear) layers, to build a simple multi-layer perceptron (MLP).

You must submit the following files to **Gradescope**:

- `hw6_q3_autograd.py`
- `hw6_q3_nn.py`
- `hw6_q3_optim.py`
- `hw6_q3_utils.py`

Do not submit the Jupyter notebook `hw6.ipynb` to Gradescope, as it will not be evaluated. The notebook is provided solely for you to test your implementation.

[Written part] (10 pt)

(a) **Manual backpropagation.**

To serve as a reference for your implementation, you will first manually perform backpropagation with a simple MLP. Consider a two-layer MLP with first layer weight $\mathbf{w} = [w_0, w_1, w_2]$, second layer weight $\mathbf{h} = [h_0, h_1]$. A ReLU activation is used for the first layer output, while no activation is used for the second layer output. Note that w_0 and h_0 are the bias terms. Each weight is initialized to 1. The dataset consists of 2 input-label pairs: $\{\mathbf{x}^{(0)} = [0, 1]^\top, y^{(0)} = 1\}$ and $\{\mathbf{x}^{(1)} = [1, 0]^\top, y^{(1)} = 0\}$. The MLP is trained on this dataset for 2 steps (1st step uses $\mathbf{x}^{(0)}$, 2nd step uses $\mathbf{x}^{(1)}$) and is evaluated using squared error $L(y, \hat{y}) = (\hat{y} - y)^2$. The neural network is optimized with gradient descent with a learning rate of 0.05.

Demonstrate your step-by-step backpropagation calculation for the first 2 training steps. Round your results up to 4 decimal digits.

- (1.5 pt) First training step: What is the gradient (derivative) for w_0, w_1, w_2, h_0, h_1 ?
- (1.5 pt) First training step: What is the updated value for w_0, w_1, w_2, h_0, h_1 ?
- (1.5 pt) Second training step: What is the gradient (derivative) for w_0, w_1, w_2, h_0, h_1 ?
- (1.5 pt) Second training step: What is the updated value for w_0, w_1, w_2, h_0, h_1 ?

(b) **Gradient of common functions.**

The sigmoid activation $\sigma(z)$ and the binary cross-entropy loss $L(y, \hat{y})$ is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{and} \quad L(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

where $\hat{y} = \sigma(f(x))$ is the probability-valued output of neural network f . In practice, the sigmoid and binary cross-entropy loss are fused into one operation for numerical stability and efficiency (i.e., `BCEWithLogitsLoss` in PyTorch). Derive:

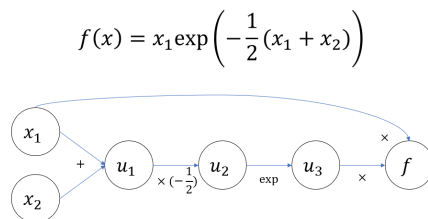
- (1.5 pt) The gradient of $\sigma(z)$ with respect to its input z .
- (2.5 pt) The simplified binary cross-entropy $L(y, \sigma(z))$ with respect to the logit $z = f(x)$ (i.e., raw, unnormalized neural network output) and the gradient of $L(y, \sigma(z))$ with respect to the z .

[Programming part] (19 pt)

(c) **Reverse-mode automatic differentiation. Implement `hw6_q4_autograd.py`.**

You are **not allowed** to use any external libraries (Numpy, PyTorch, etc.) for this question. You are **allowed** to use the provided numerically stable `sigmoid` and `log_sigmoid` functions.

For a simple 2-layer MLP, we already have to perform lots of manual computation. In popular neural network frameworks, this gradient computation is usually handled by reverse-mode automatic differentiation. These engines perform gradient calculation by maintaining a "computation graph" (represented by a directed acyclic graph (DAG)) that records all operations that lead to the output.



Consider the root node as f , we can do a depth-first traversal and obtain the following order $x_1, x_2, u_1, u_2, u_3, f$. Reversing this order to obtain the backpropagation order $f, u_3, u_2, u_1, x_2, x_1$. We will implement our own automatic differentiation module by chopping the neural network into its smallest unit, which is an individual scalar. The graph traversal mechanism has been provided. You would only need to implement the **forward** and **backward** methods for each operation.

- (d) **Neural network. Implement `hw6_q3_nn.py` and `hw6_q3_optim.py`.**

You are **not allowed** to use any external libraries (Numpy, PyTorch, etc.) for this question.

- (e) **Fit everything together. Implement `hw6_q4_utils.py`.**

A Jupyter notebook `hw6.ipynb` is provided for you to test your implementation against the PyTorch implementation. Your results should be very close to the PyTorch implementation.

Solution.